

FRED TALKS

29th May 2026

CONTENTS

The lies I used to tell myself

CATE HALL · 1900 WORDS

Semantic Search Without Embeddings

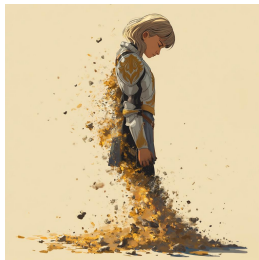
DOUG TURNBULL · 3143 WORDS

AI makes weak engineers less harmful

SEANGOEDECKE.COM RSS FEED · 1085 WORDS

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · [SOURCE](#)



1. What doesn't kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We'd had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can't say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn't want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There's a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don't always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great [“50 Things” style post](#) a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: “Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless.”

I believed this completely, once. I’ve since come to believe that what’s unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it’s not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it’s not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I “just knew” with my first husband. I wasn’t sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There’s a belief, especially common among wonky types, that you don’t find \$20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it’s actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they’re missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don’t assume one exists. Sometimes the answer really is just inertia or fear.

4. If it’s worth doing, it’s worth doing well.

Almost nothing is worth doing “well.” Many things can and should be done to a minimal standard of quality, because anything more is a [waste of effort](#) that can be better allocated. The exceptions aren’t things that should be done “well,” but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really “do everything the way they do anything” — whether that’s in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter ... and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. "All people are insane. They will do anything at any time, and God help anybody who looks for reasons."

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they'd do next, seemed like a fool's errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn't fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People's behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what's rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can't buy happiness.

The good version of this advice is: You'll still have to work at being happy, even if you're rich. Money in your bank account won't do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there's research showing money doesn't buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why “wisdom is knowledge that can't be transmitted.” Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, *You Can Just Do Things*, is available for purchase.

Semantic Search Without Embeddings

DOUG TURNBULL · 08 JAN 2026 · [SOURCE](#)

When I need to stay warm, I search for “long johns” or “long underwear”. But modern outdoors clothing stores label this a “base layer”. For a waterproof jacket I search for “slickers” or a “ski jacket”, not realizing what I should search for is a “shell.”

Despite my outdated terminology, search still works. I'm somehow understood and shown the right content.

We call this **semantic search**. When you hear that, you might think embeddings. Today I want to stretch your thinking beyond. Fantasize and semanticize.

Some teams wrongly assume semantic search equates to only “vector search”. You have more options, some that might better align to your domain and existing organizational skills. LLMs simplify classic ways of organizing search queries and information.

Let's contrast different approaches, and maybe you'll see there's another approach that might be a better fit for you.

Semantic search - what exactly is it?

In semantic search, content and query map to a *shared representation*. This space has a *similarity* function that scores similar items higher than less similar.

Let's walk through an example.

A user searching “round red fruit that grows on trees” probably wants an apple.

Humans know this - search engines don't. So we need to help.

We map the query to three properties: [round, red, fruit]

In our search index we've also mapped three items into that space:

An **apple**: [round, red, fruit]

An **orange**: [round, orange, fruit]

A **baseball**: [round, white, ball]

Now we have a shared *representation*: a three tuple of (shape, color, item type).

How do we measure similarity? For each property (shape, color, item type) we could score a “1” if they're equal, otherwise a 0. Then add to get a total score:

Similarity(query, apple) = 1 + 1 + 1 = 3

Similarity(query, orange) = 1 + 0 + 1 = 2

Similarity(query, baseball) = 1 + 0 + 0 = 1

So the user sees search results

1. Apple (Score 3)
2. Orange (Score 2)
3. Baseball (Score 1)

Ok, so a fairly silly method of semantic search. It ignores properties that matter (size, for example). It requires us to comprehensively tag everything. Still many teams try to tag + synonym their way to better search, with middling results.

Embeddings for semantic search

The default answer to the “semantic search” question is embeddings. The Internet is littered with documentation for [how vector search works](#). I’ll do the speedrun here ([references here](#)).

We can learn a representation, an embedding, for apple and orange so that `similarity(apple, orange) > similarity(apple, baseball)`.

We learn an embedding from training data. We might declare when items get clicked for the same query, they are similar. We notice both “apple” and “orange” get clicked for query “fruit that grows on trees” at high frequency:

[fruit that grows on trees 🔍]

1. 🍎 (clicked)
2. 🥕
3. 🍊 (clicked)

We start with a random vector for each. A list of random floating points numbers like `[0.01, -0.1, 0.05]`. Then we train, in this super-secret-ultra-high-tech algorithm

```
for session in sessions:
    together = []
    for lhs in session: # ie the apple
        for rhs in session: # ie the orange
            if lhs != rhs:
                nudge_closer(lhs, rhs) # ie nudge a
```

(Not shown: nudging away the negative, dissimilar items)

After training, for many queries+sessions, maybe apple becomes `[0.1, -0.2, 0.05]` while orange becomes a similar vector: `[0.09, -0.21, 0.04]`. We measure the similarity with [cosine similarity](#), [euclidean distance](#), and others. (We can replace “shared search session clicks” with any other notions of “similar”: occurs in the same text passages, both occur in shopping carts, etc).

Learning an embedding per item presents problems. For example, maybe we just added strawberry 🍓 to our product listings. With no training data, the vector `strawberry_vector = embeddings['strawberry']` initializes to random garbage. It won’t have a meaningful value until we’ve shown it to many users.

We'd prefer to compute embeddings from **features** of the item. Then strawberry 🍓 could get an embedding right away. IE instead of `strawberry_embedding = embeddings['strawberry']` we can do `strawberry_embedding = f(heart_shaped, red, fruit)`. We build an **encoder**. Given the item's attributes we compute embeddings that produce the expected similarity. Embedding models downloaded from hugging face specialize in encoding specific type/domain of content for a specific task (search, etc).

We create encoders for all kinds of features: ordinal values (like a list of shapes, colors), the raw title / description text itself, images and so on. We can also train a two-tower model: one encoder for queries - ie "round fruit that goes on trees" - and another for documents.

Like other machine learning models we tradeoff precision for generality. We need the right features, we need enough training data for all the variations in those features (curse of dimensionality) and so on.

The missing semantic search piece of vector retrieval

One pain point comes up right away with embeddings: lack of *matching*.

How would users actually react to these search results?

[fruit that grows on trees 🔍]

1. 🍏 Apple
2. 🍊 Orange
3. ⚾ Baseball.

In my experience, negatively.

It may have the perfect ranking. But search should exclude items outside what the user intended. They find fruit acceptable. But not baseballs.

So earlier I lied. What users actually want from semantic search:

- **Representation** - some space both queries and content can share representation
- **A similarity function** - A way of measuring how near / far items in the shared representation are from each other
- **Match criteria (new!)** - A system where an item can be said to be "match" or not for the query in the semantic space. It is the thing, or its not the thing.

In other words, I gave an incomplete **semantic search definition**. Embeddings thrive on the first 2, but don't do a great job at including / excluding results.

Understanding users (and their queries) matters a great deal in search. That requires including / excluding appropriate items based on terminology and categories familiar to users.

You might think "I'll just find a cutoff in my embedding similarity". Sadly, there's not a magical "at 0.8 similarity it's not a match". In some domains / queries one threshold works. In another, a very different threshold. Further, thresholding becomes tougher when users give multiple criteria. An embedding model might rank blood oranges above green apples for query "red apple". What matters more, the color or item type in the similarity?

Thresholds help exclude some obviously irrelevant results and improve performance, but they don't use domain-specific criteria, in the user's language, to include / exclude results.

The other semantic search approach (hierarchical managed taxonomies)

One system tries to solve all three problems: *Managed vocabularies or taxonomies*.

Taxonomies map queries + content to a hierarchy of concepts, using the language of the domain. Think a neatly organized directory tree. Or the Dewey Decimal System

Here's an example from the Wayfair WANDS furniture dataset of their category tree; Where you'd categorize "novelty rocking horses" apparently:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

We can take a query for **hobby horse**. A human, curating a bunch of query rules, could manually map this to a node:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

Assuming the corpus categorizes products correctly, we can search, filter, boost, etc using this.

We have a *representation* (the category tree), do we have a similarity function?

Yes, we can rank as follows:

1. Direct matches Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

2. Parents / siblings. Baby & Kids / Toddler & Kids Playroom / Indoor Play / Dollhouses

3. Those rank higher than grandparents / cousins (`Baby & Kids / Toddler & Kids Playroom / Beanbag Chairs / ...`)

We can define meaningful match criteria too. We include shared parent categories (ie siblings). But exclude anything beyond cousins / shared grandparents as needed.

Many teams already maintain systems for organizing and cataloging information. For many domains, a good taxonomy can be a moat. Especially for technical domains (legal, medical, fashion, finance) where precise categorization really matters. Existing taxonomies already exists, and statistical fuzziness of embeddings can create huge downside.

In the past, the management of a taxonomy would be massive. It might require complex rules, mapping of query phrases to taxonomy nodes, and a team of labelers and experts to keep everything straight. This has long been an appeal of embeddings.

But LLMs sneakily supercharge these old school ways of organizing information. Given a taxonomy, LLMs can easily + cheaply classify products and queries. It's more approachable to "fine tune". AI augmented labeling can simplify management of knowledge making it more approachable.

Semantic similarity doesn't need a vector index

Above I described a similarity function above (direct matches, then parents, then grandparents). With some creative tokenization, this similarity can happen in a normal BM25 index.

Consider our query for **hobby horse**. We know this maps to this node:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

How do we score content in such a way so that direct matches rank higher than siblings, and so on?

We need is a hierarchical tokenizer. A function that produces the full path of every parent directory:

```
In: hierarchical_tokenizer("Baby & Kids / Toddler & Kids Playroom / I
Out: ['Baby & Kids',
      'Baby & Kids / Toddler & Kids Playroom',
```

```
'Baby & Kids / Toddler & Kids Playroom / Indoor Play',  
'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

If you index using this tokenizer, you'll recreate scoring needed for the direct > parent > grandparent similarity function I described earlier.

The key reason: BM25 rewards a rare match over common match.

Root nodes (ancestors) have higher document frequency, they're common. Common matches score lower in BM25.

That's because every item labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses` tokenized hierarchically has a `Baby & Kids` term. So does a document labeled `Baby & Kids / Kids Furniture ... / Beanbag Chairs`. `Baby & Kids` happens all over the place - maybe like 10% of the index.

Child nodes have low document frequency, they're rare. In BM25, rare, more specific matches outrank common ones.

A product labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditonal Rocking Horses` when tokenized will include the full path in the token. There might be 10 products in the entire index in this exact category. So that rareness - that high specificity - shoots this narrower, closer match to the top.

To do a deeper walkthrough

When I come along and search for

```
In: hierarchical_tokenizer(  
    "Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking H  
)  
Out: ['Baby & Kids',  
      'Baby & Kids / Toddler & Kids Playroom',  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play',  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

I'm searching for five tokens now, OR'd (really summed):

```
"Baby & Kids" OR  
"Baby & Kids / Toddler & Kids Playroom" OR
```

"Baby & Kids / Toddler & Kids Playroom / Indoor Play" OR
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses"
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses"

Each token of my query has a job. It matches different levels of specificity, and the closer we get the full query classification tree, the higher the score.

Search Term	Job	Score (due to doc freq)	Example of matches
"Baby & Kids"	Matches highest level category	Lowest	Baby & Kids / Toddler & Kids Bedroom Furniture / Kids Headboards
"Baby & Kids / Toddler & Kids Playroom"	Matches next most specific category Getting to "play" stuff for kids, better than just kid stuff. 👍	Low	Baby & Kids / Toddler & Kids Playroom / Playroom Furniture / Baby Gyms & Playmats / Folding Baby Gyms & Playmats
...	...	...	...
Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses	Matches a very, narrow specific category. Horses! 🐎 Even if not "Novelty Rocking Horses" its a defensible "match" to the user.	High	Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditonal Rocking Horses

While we keep things in horse → toys → playroom → kids order, we haven't solved the 3rd problem. Limiting matches.

That one's easy. At query time, you can decide how much to limit retrieval. Perhaps you only query the direct node and one parent:

"Baby & Kids / ... / Rocking Horses" OR
"Baby & Kids / ... / Rocking Horses / Novelty Rocking Horses"

You can tighten / relax in categories that matter to users, *semantically*, up a layer to still stay within the realm of "kid play".

If the taxonomy makes sense to users, if it reflects search behaviors (big ifs), they'll understand why you include / exclude certain results.

Importantly, this becomes just ONE component of the ranking / filtering. We can still have other ingredients we can use:

- Keyword matches (for exact mention of properties in titles + description)
- Embedding search (perhaps within the category matches)
- Taxonomies for other entities (colors, materials, etc)

Building a taxonomy: start simple and evolve

Building and maintaining a good taxonomy is no joke. That's why there's a field of taxonomists you can hire. And I don't want to elide the tradeoffs with embeddings here. It's work!

A taxonomy doesn't need to be as deep / complex as what I've laid out. You can start simple. Ask an LLM. Here's one with color.

Take the 7 primary colors. Then nested under each, create subtypes of each color.
Generate a list like a directory trees, in the form PRIMARY / SECONDARY

RED / Crimson

RED / Scarlet

RED / Burgundy

RED / Maroon

RED / Brick Red

RED / Cherry

ORANGE / Tangerine

ORANGE / Amber

ORANGE / Coral

ORANGE / Burnt Orange

ORANGE / Peach

ORANGE / Apricot

...

It's a start.

Generally start simple. Broad, high level, top level categories like Furniture or Baby & Kids. Then eventually you realize this category has become too large and too diverse. You might see a natural in the content split:

Baby & Kids / Toddler & Kids Playroom

Baby & Kids / Toddler & Kids Bedroom Furniture

Eventually you get to a point where the distinction between Novelty Rocking Horse and Classic Rocking Horse matters for some reason.

You may see a flaw: an assumption each product sits within a single category. That categories are mutually exclusive. Reality, of course, is messy: some products may sit between categories. You can imagine a small goofy looking couch that could reasonably tagged under Baby & Kids / Toddler & Kids Bedroom Furniture but also Living Room / Loveseats. In Fashion, some activewear might also be underwear. And what about athleisure?

One solution could be to double categorize an item. Give it both classifications - with some weight preferring one categorization over another.

A final concern - there's a big difference between the official pure way of classifying furniture and how a search user might categorize the products. The exacting, furniture-expert taxonomy might have very sophisticated (and accurate) ways to classify furniture. A messier classification that matches how users think about things might be more useful in the end.

Classifying into taxonomies has never been easier

Building taxonomy classifiers with LLMs - something I've built an [entire class for](#), and could be a post on its own. But lets do a quick walkthrough of some simpler approaches.

Embeddings actually help us here. The sweet spot of embeddings in search might be building better classifiers, not in direct ranking and retrieval.

Perhaps our products fit in a few thousand unique categories. Imagine we encode an embedding each of these in a small in-memory array of vectors. We'll use it to lookup categories for our queries.

Now take a query "hobby horse". Encode the query into an embedding, and find the most similar category embedding. Maybe this retrieves Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses.

Not accurate enough? Let's take it up a notch.

We could ask an LLM to hallucinate a fake label before categorizing. Ask an LLM the following:

Be creative and hallucinate a set of classifications for the query below that look like the real classifications.

Product classifications might look like:

```
'Furniture / Living Room Furniture / Coffee Tables & End Tables / Coffee  
'Décor & Pillows / Decorative Pillows & Blankets / Throw Pillows'  
'Furniture / Bedroom Furniture / Dressers & Chests'  
'Outdoor / Outdoor & Patio Furniture / Patio Furniture Sets / Patio  
'Home Improvement / Bathroom Remodel & Bathroom Fixtures / Bathroom  
'Lighting / Wall Lights / Bathroom Vanity Lighting'  
'Kitchen & Tabletop / Kitchen Organization / Food Storage & Canisters  
'School Furniture and Supplies / School Furniture / School Chairs &  
'Baby & Kids / Toddler & Kids Bedroom Furniture / Kids Beds',
```

If you feel inspired, return many unique values in a list. Be creative. Cast a wide net with related but diverse categories.

Here's the query to invent categories for:

hobby horse

When you actively want hallucinations, you can get by with very small models. You're not relying on the LLM for accuracy, just creativity and close-enough language generation.

Now let's say you do this. It gives you a classification like

```
'Baby & Kids / Toys / Pretend Play & Dress Up / Pretend Play Toys / Horse'
```

More descriptive, closer to the language actually used in the classifications. More likely to give back a real, high quality classification.

That's just a taste. The modern landscape for building large multi-label classifiers has gotten easier than ever thanks to LLMs and embeddings.

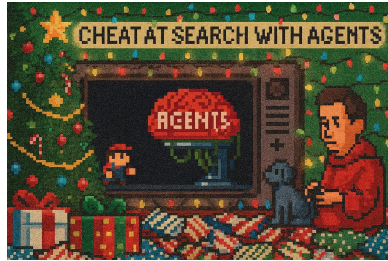
Final thoughts

Embeddings are great. But you don't need embeddings to build "semantic search". In fact, it can be counter productive for problems requiring exacting, precise matching. As I've argued, embeddings are the wrong lens to think about the problem.

Maybe you don't need to rebuild everything from scratch, but instead leverage what you're good at. With the addition of LLMs. Don't apologize for living with a more organized, approach to semantic search.

Enjoy software.doug in training course form!

20% off through December 19, 2025 with code ps5



I hope you join me at [Cheat at Search with LLMs](#) to learn how to apply LLMs to search applications. Check out [this post](#) for a sneak preview.

AI makes weak engineers less harmful

SEANGOEDECKE.COM RSS FEED · 08 MAY 2026 · [SOURCE](#)

Like other kinds of puzzle-solving, software engineering ability is strongly heavy-tailed. The strongest engineers produce way more useful output than the average, and the weakest engineers often are actively net-negative: instead of moving projects along, they create problems that their colleagues have to spend time solving. That's why many tech companies try to [build](#) a small, ludicrously well-paid team instead of a large team of more average engineers, and so far this seems to be a winning strategy.

Being effective in a large tech company is often about managing this phenomenon: trying to arrange things so that the most competent people land on projects you want to succeed, and the least competent are shunted out of the way¹. For instance, if you're technical lead on a project,

you more or less have to ensure² that the most critical pieces are in the hands of people who won't screw them up (whether by directly assigning the work, or by making sure someone can "sit on the shoulder" of the engineer who you're worried about).

Claude Code changed this. Frontier LLMs don't have the taste or the system familiarity of a strong engineer, but they have absolutely raised the floor for weak engineers. Instead of getting a pull request that could never possibly work or would cause immediate problems, the worst you'll now see is a standard LLM pull request: wrong in some ways, baffling in others, but at least functional on the line-by-line level and not so obviously incorrect that someone with no knowledge of the codebase could point it out. That is a huge improvement!

You can try this out yourself. If you attempt to deliberately make mistakes while working with a coding agent, you'll find that the agent pushes back hard against many obvious errors (i.e. caching user data with a non-user-specific key, writing an infinite loop that might never terminate, or leaking open files). Of course, the agent will still miss subtle errors, particularly ones that require understanding other parts of the codebase.

Working with the least effective engineers is now sometimes like working with a Claude Opus or Codex instance that you communicate with over Slack. Occasionally it's *literally* that: your colleague is simply pasting your messages into Claude Code and pasting you the response. This is annoying, but it's a much better experience than working with this kind of engineer directly. After all, you probably already work with a bunch of LLM instances. The Slack interface is not ideal - unlike using Claude Code directly, you sometimes wait hours or days for a response, and you don't get visibility into the agent's thought processes - but it's still helpful on the margin. More compute being thrown at your problem is better than less.

Of course, this isn't a great state of affairs for the engineer in question, who is almost certainly learning less than if they were making their own (bad) decisions. It's also a bad state of affairs for the company, who is paying a human salary and getting a Copilot subscription (which they're likely also paying for)³. After the current push to figure out what value AI is adding to engineers, I suspect there will be a push to figure out what value *engineers are adding to AI*, and the engineers who aren't adding much may find themselves out of a job.

You can't talk to Claude-over-Slack like you'd talk to normal Claude. If you tend to handle LLMs roughly (insulting them, or just being very curt), you'll have to change your communication style. A human is going to read your messages, after all, even if you're really interacting with a LLM. There's no point being rude. But if, like me, you say please-and-thank-you to the models⁴, you can treat your LLM-using coworker as just another Copilot window or Codex tab. It's far better than having to treat them as an unwitting saboteur.

Not all net-negative engineers use AI tools like this. Many are strongly convinced in their own wrong opinions about how to build good software, or mistrust AI in general, or believe that relying heavily on LLMs is not a good way to improve⁵. But *no* strong engineers use AI tools like this. Even when they're being lazy or sloppy, a capable engineer will have enough baseline taste to catch obvious AI-generated errors. So the phenomenon of engineers⁶ becoming thin wrappers around Claude Code is limited to the kind of engineers for whom this is an improvement in their work product.

1. More charitably: many "least competent" engineers are just out of their comfort zone, and can be fine or even excel under the right circumstances (though in my view the best engineers are able to do good work in a wide variety of environments). Also, I don't currently work with a lot of incompetent people. Much of this is based on past experience or talking to other engineers in the industry.

↩

2. Since your managers are doing the same thing, this can sometimes feel like Moneyball: you're trying to identify underappreciated talent who are strong enough to help you win without being so high-profile that your boss poaches them to lead something else.

↩

3. I suppose it's better to pay for nothing than to pay for net-negative output, but it still doesn't seem *good*.

↩

4. I think this is actually the right way to hold Claude Opus 4.7.

↩

5. Is this true? I think relying on LLMs is not a great way for most engineers to improve, but if LLM output is consistently better than your own, it might be different. So long as you're paying attention to where the LLM does better, it could actually be a good way to learn.

↩

6. I don't have as much experience (or anecdotes) about non-engineers falling into this trap, but [this post](#) has convinced me that it might be worse.

↩